Bakery Business

Database Design Document

PROJECT OVERVIEW

The purpose of this project is to design a relational database for a small bakery business owned by a sole proprietor. The database will serve as a centralised system for storing data about the business's operations, enabling data-driven decision making in the future.

PROBLEM STATEMENT

This database addresses the following challenges:

- Providing an effective and organised storage system for business records
- Ensuring data is relevant, complete, consistent, and of high quality over time
- Creating a reliable data foundation for future business decisions and analysis

REQUIREMENT ANALYSIS

Functional requirement

- Store information about products, orders, customers, ingredients, and product categories
- Support easy data entry, update, and retrieval across all tables
- Track ingredient usage per product (recipe management)
- Record ingredient purchases for inventory tracking

Non-functional requirement

- Ensure data integrity through constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK, UNIQUE)
- Maintain data quality through controlled fields (ENUM, DEFAULT values)
- Design for scalability to accommodate business growth

DATA MODELING

Conceptual Model

Entities:

Product, Product Category, Order, Order Details, Customer, Ingredient, Product Ingredient, Purchases

Relationships:

Customer --- Order ---Product ---Product category

Order details – Order

Ingredient --- Product Ingredient

Purchases --- Ingredient

Logical Model

Entity	Key Attributes	Unique Identifier
Product	Product_name, Category_ID, Unit price	Product_ID
Product category	Category_name,	Category_ID
Order	Customer_id, order_date, delivery_date, payment_mode, delivery_mode	Order_ID
OrderDetails	Order_ID, Product_ID, quantity, discount, Unit_price, total_amount	OrderDetails_ID
Customer	Customer_name, gender, phone_number, email, location, joined_date, customer_type	Customer_ID
Ingredients	Ingredient_name	Ingredient_ID
Product_Ingredient	Ingredient_ID, product_id, ingredient_quantity, product_quantity, production_date	Product_ Ingredient_ID
Purchases	Purchase_date, Ingredient_ID, ingredient_unit_price, Ingredient_quantity, Ingredient_size	Purchases _ID

Cardinality

Entity A	**Cardinality**	**Entity B**
Customer	1:N	Order
Order	1:N	OrderDetails
OrderDetails	N:1	Product
Product	N:1	Product Category
Ingredient	1:N	Product_Ingredient
Ingredient	1:N	Product
Ingredient	1:N	Purchases

DATABASE SCHEMA

Product Table

Attribute	Description
Product_ID	Unique identifier for every product VARCHAR(20), PRIMARY KEY
Product_name	Name of the product VARCHAR(50), NOT NULL
Category_ID	Category the product belongs to VARCHAR(20), FOREIGN KEY → ProductCategory
Unit price	Price of a single product DECIMAL(10,2), CHECK (unit_price> 0), NOT NULL

Product Category Table

Attribute	Description
category_ID	Unique identifier for every category VARCHAR(20), PRIMARY KEY
category_name	Name of the category VARCHAR(50), NOT NULL

Order Table

Attribute	Description
Order_ID	Unique identifier for every order INT, PRIMARY KEY
Customer_ID	Identifier of the customer who placed the order VARCHAR(20), FOREIGN KEY → Customer
Order_date	Date and time the order was placed DATETIME, NOT NULL, DEFAULT CURRENT_TIMESTAMP
Delivery date	Date the order was or will be delivered DATE, NOT NULL
Payment_mode	Mode of payment by the customer ENUM ('Cash', 'Mobile Money', 'Card'), NOT NULL
Delivery_mode	Mode the order was delivered ENUM ('Pickup', 'Delivery'), NOT NULL

OrderDetails Table

Attribute	Description
OrderDetails_ID	Unique identifier for every order detail INT, PRIMARY KEY
Order_ID	Identifier of the parent order INT, FOREIGN KEY → Order
Product_ID	Identifier of the product ordered VARCHAR(20), FOREIGN KEY → Product
Unit_price	Price of the product at time of order (snapshot) DECIMAL(10,2), CHECK > 0, NOT NULL
quantity	Number of units ordered INT, CHECK > 0, NOT NULL
Discount	Discount applied to the product (%) DECIMAL(5,2), NOT NULL, DEFAULT 0
Total amount	Total amount paid for this line DECIMAL(10,2), NOT NULL Formula: Quantity × Unit_price × (1 − Discount / 100)

Customer Table

Attribute	Description
Customer_ID	Unique identifier for every customer VARCHAR(20), PRIMARY KEY
Customer_name	Full name of the customer VARCHAR(100)
gender	Gender of the customer ENUM('Male', 'Female', 'Other')
Phone_number	Contact number of the customer VARCHAR(20)
Email	Email address of the customer VARCHAR(50), UNIQUE
Location	Address or area of the customer VARCHAR(100)
Joined_date	Date and time the customer first placed an order DATETIME, DEFAULT CURRENT_TIMESTAMP
Customer_type	Customer classification (e.g. Retail, Wholesale, Regular) VARCHAR(50), NOT NULL

Ingredient Table

Attribute	Description
Ingredient_ID	Unique identifier for every ingredient VARCHAR(20), PRIMARY KEY
Ingredient_name	Name of the ingredient VARCHAR(100), NOT NULL

Product_Ingredient Table

Attribute	Description
pi_id	Unique identifier for every recipe entry TABLE PRIMARY KEY
Ingredient_ID	Identifier of the ingredient used VARCHAR(20), FOREIGN KEY → Ingredient
Product_id	Identifier of the product being made VARCHAR(20), FOREIGN KEY → Product
Ingredient_quantity	Amount of ingredient used INT, CHECK > 0, NOT NULL
Product_quantity	Number of product units produced INT, CHECK > 0, NOT NULL
production_date	Date the production was made DATE, DEFAULT CURRENT_DATE, NOT NULL

Purchases Table

Attribute	Description
Purchases_ID	Unique identifier for every purchase record VARCHAR(20), PRIMARY KEY
Ingredient_ID	Identifier of the purchased ingredient VARCHAR(20), FOREIGN KEY → Ingredient
Ingredient_size	Size or weight of the ingredient purchased DECIMAL(10,3)

Ingredient_unit_price	Unit price of the ingredient at time of purchase DECIMAL(10,2), NOT NULL
Ingredient_quantity	Number of units purchased INT, CHECK > 0, NOT NULL
Purchase_date	Date the ingredient was purchased DATETIME DEFAULT CURRENT_TIMESTAMP

ADDITIONAL CONSTRAINTS

Composite Key - Order Table

A composite key on (Order_ID, Customer_ID, Order_date) ensures that every order record entered into the Order table is unique.

Unique Constraint - OrderDetails Table

A UNIQUE constraint on (Order_ID, Product_ID) ensures that the same product cannot appear more than once on a single order line, preserving data integrity.